

Introduction to Python Programming

2



Objectives

In this chapter, you'll:

- Continue using IPython interactive mode to enter code snippets and see their results immediately.
- Write simple Python statements and scripts.
- Create variables to store data for later use.
- Become familiar with built-in data types.
- Use arithmetic operators and comparison operators, and understand their precedence.
- Use single-, double- and triple-quoted strings.
- Use built-in function `print` to display text.
- Use built-in function `input` to prompt the user to enter data at the keyboard and get that data for use in the program.
- Convert text to integer values with built-in function `int`.
- Use comparison operators and the `if` statement to decide whether to execute a statement or group of statements.
- Learn about objects and Python's dynamic typing.
- Use built-in function `type` to get an object's type.

2.1 Introduction	2.7 Decision Making: The <code>if</code> Statement and Comparison Operators
2.2 Variables and Assignment Statements	2.8 Objects and Dynamic Typing
2.3 Arithmetic	2.9 Intro to Data Science: Basic Descriptive Statistics
2.4 Function <code>print</code> and an Intro to Single- and Double-Quoted Strings	2.10 Wrap-Up Exercises
2.5 Triple-Quoted Strings	
2.6 Getting Input from the User	

2.1 Introduction

In this chapter, we introduce Python programming and present examples illustrating key language features. We assume you’ve read the IPython Test-Drive in Chapter 1, which introduced the IPython interpreter and used it to evaluate simple arithmetic expressions.

2.2 Variables and Assignment Statements

You’ve used IPython’s interactive mode as a calculator with expressions such as

```
In [1]: 45 + 72
Out[1]: 117
```

As in algebra, Python expressions also may contain **variables**, which store values for later use in your code. Let’s create a variable named `x` that stores the integer 7, which is the variable’s **value**:

```
In [2]: x = 7
```

Snippet [2] is a **statement**. Each statement specifies a task to perform. The preceding statement creates `x` and uses the **assignment symbol** (`=`) to give `x` a value. The entire statement is an **assignment statement** that we read as “`x` is assigned the value 7.” Most statements stop at the end of the line, though it’s possible for statements to span more than one line. The following statement creates the variable `y` and assigns to it the value 3:

```
In [3]: y = 3
```

Adding Variable Values and Viewing the Result

You can now use the values of `x` and `y` in expressions:

```
In [4]: x + y
Out[4]: 10
```

The `+` symbol is the **addition operator**. It’s a **binary operator** because it has *two* **operands** (in this case, the variables `x` and `y`) on which it performs its operation.

Calculations in Assignment Statements

You’ll often save calculation results for later use. The following assignment statement adds the values of variables `x` and `y` and assigns the result to the variable `total`, which we then display:

```
In [5]: total = x + y

In [6]: total
Out[6]: 10
```

Snippet [5] is read, “total is assigned the value of $x + y$.” The = symbol is not an operator. The right side of the = symbol always executes first, then the result is assigned to the variable on the symbol’s left side.

Python Style

The *Style Guide for Python Code*¹ helps you write code that conforms to Python’s coding conventions. The style guide recommends inserting one space on each side of the assignment symbol = and binary operators like + to make programs more readable.

Variable Names

A variable name, such as `x`, is an **identifier**. Each identifier may consist of letters, digits and underscores (`_`) but may not begin with a digit. Python is **case sensitive**, so `number` and `Number` are *different* identifiers because one begins with a lowercase letter and the other begins with an uppercase letter.

Types

Each value in Python has a **type** that indicates the kind of data the value represents. You can view a value’s type, as in:

```
In [7]: type(x)
Out[7]: int

In [8]: type(10.5)
Out[8]: float
```

The variable `x` contains the integer value 7 (from snippet [2]), so Python displays `int` (short for integer). The value 10.5 is a **floating-point number** (that is, a number with a decimal point), so Python displays `float`.

Python’s **type built-in function** determines a value’s type. A **function** performs a task when you **call** it by writing its name, followed by parentheses, `()`. The parentheses contain the function’s **argument**—the data that the type function needs to perform its task. You’ll create *custom* functions in later chapters.

✓ Self Check

1 (True/False) The following are valid variable names: `3g`, `87` and `score_4`.

Answer: False. Because they begin with a digit, `3g` and `87` are invalid names.

2 (True/False) Python treats `y` and `Y` as the same identifier.

Answer: False. Python is case sensitive, so `y` and `Y` are different identifiers.

3 (IPython Session) Calculate the sum of 10.8, 12.2 and 0.2, store it in the variable `total`, then display `total`’s value.

Answer:

```
In [1]: total = 10.8 + 12.2 + 0.2

In [2]: total
Out[2]: 23.1
```

1. <https://www.python.org/dev/peps/pep-0008/>.

2.3 Arithmetic

Many programs perform arithmetic calculations. The following table summarizes the **arithmetic operators**, which include some symbols not used in algebra.

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	$b \cdot m$	<code>b * m</code>
Exponentiation	**	x^y	<code>x ** y</code>
True division	/	x/y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Floor division	//	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or $\lfloor x \div y \rfloor$	<code>x // y</code>
Remainder (modulo)	%	$r \bmod s$	<code>r % s</code>

Multiplication (*)

Rather than algebra's center dot ($\cdot$), Python uses the **asterisk (*) multiplication operator**:

```
In [1]: 7 * 4
Out[1]: 28
```

Exponentiation (**)

The **exponentiation (**) operator** raises one value to the power of another:

```
In [2]: 2 ** 10
Out[2]: 1024
```

To calculate the square root, you can use the exponent 1/2 (that is, 0.5):

```
In [3]: 9 ** (1 / 2)
Out[3]: 3.0
```

True Division (/) vs. Floor Division (//)

True division (/) divides a numerator by a denominator and yields a floating-point number with a decimal point, as in:

```
In [4]: 7 / 4
Out[4]: 1.75
```

Floor division (//) divides a numerator by a denominator, yielding the highest *integer* that's not greater than the result. Python **truncates** (discards) the fractional part:

```
In [5]: 7 // 4
Out[5]: 1
```

```
In [6]: 3 // 5
Out[6]: 0
```

```
In [7]: 14 // 7
Out[7]: 2
```

In true division, -13 divided by 4 gives -3.25:

```
In [8]: -13 / 4
Out[8]: -3.25
```

Floor division gives the closest integer that's *not greater than* -3.25—which is -4:

```
In [9]: -13 // 4
Out[9]: -4
```

Exceptions and Tracebacks

Dividing by zero with / or // is not allowed and results in an **exception**—a sign that a problem occurred:

```
In [10]: 123 / 0
-----
ZeroDivisionError                                Traceback (most recent call last)
<ipython-input-10-cd759d3fcf39> in <module>()
----> 1 123 / 0

ZeroDivisionError: division by zero
```

Python reports an exception with a **traceback**. This traceback indicates that an exception of type `ZeroDivisionError` occurred—most exception names end with `Error`. In interactive mode, the snippet number that caused the exception is specified by the 10 in the line

```
<ipython-input-10-cd759d3fcf39> in <module>()
```

The line that begins with `----> 1` shows the code that caused the exception. Sometimes snippets have more than one line of code—the 1 to the right of `---->` indicates that line 1 within the snippet caused the exception. The last line shows the exception that occurred, followed by a colon (`:`) and an error message with more information about the exception:

```
ZeroDivisionError: division by zero
```

The “Files and Exceptions” chapter discusses exceptions in detail.

An exception also occurs if you try to use a variable that you have not yet created. The following snippet tries to add 7 to the undefined variable `z`, resulting in a `NameError`:

```
In [11]: z + 7
-----
NameError                                Traceback (most recent call last)
<ipython-input-11-f2cdbf4fe75d> in <module>()
----> 1 z + 7

NameError: name 'z' is not defined
```

Remainder Operator

Python's **remainder operator** (`%`) yields the remainder after the left operand is divided by the right operand:

```
In [12]: 17 % 5
Out[12]: 2
```

In this case, 17 divided by 5 yields a quotient of 3 and a remainder of 2. This operator is most commonly used with integers, but also can be used with other numeric types:

```
In [13]: 7.5 % 3.5
Out[13]: 0.5
```

In the exercises, we use the remainder operator for applications such as determining whether one number is a multiple of another—a special case of this is determining whether a number is odd or even.

Straight-Line Form

Algebraic notations such as

$$\frac{a}{b}$$

generally are not acceptable to compilers or interpreters. For this reason, algebraic expressions must be typed in **straight-line form** using Python’s operators. The expression above must be written as `a / b` (or `a // b` for floor division) so that all operators and operands appear in a horizontal straight line.

Grouping Expressions with Parentheses

Parentheses group Python expressions, as they do in algebraic expressions. For example, the following code multiplies 10 times the quantity `5 + 3`:

```
In [14]: 10 * (5 + 3)
Out[14]: 80
```

Without these parentheses, the result is *different*:

```
In [15]: 10 * 5 + 3
Out[15]: 53
```

The parentheses are **redundant** (unnecessary) if removing them yields the *same* result.

Operator Precedence Rules

Python applies the operators in arithmetic expressions according to the following **rules of operator precedence**. These are generally the same as those in algebra:

1. Expressions in parentheses evaluate first, so parentheses may force the order of evaluation to occur in any sequence you desire. Parentheses have the highest level of precedence. In expressions with **nested parentheses**, such as `(a / (b - c))`, the expression in the *innermost* parentheses (that is, `b - c`) evaluates first.
2. Exponentiation operations evaluate next. If an expression contains several exponentiation operations, Python applies them from right to left.
3. Multiplication, division and modulus operations evaluate next. If an expression contains several multiplication, true-division, floor-division and modulus operations, Python applies them from left to right. Multiplication, division and modulus are “on the same level of precedence.”
4. Addition and subtraction operations evaluate last. If an expression contains several addition and subtraction operations, Python applies them from left to right. Addition and subtraction also have the same level of precedence.

We’ll expand these rules as we introduce other operators. For the complete list of operators and their precedence (in lowest-to-highest order), see

<https://docs.python.org/3/reference/expressions.html#operator-precedence>

Operator Grouping

When we say that Python applies certain operators from left to right, we are referring to the operators' **grouping**. For example, in the expression

$$a + b + c$$

the addition operators (+) group from left to right as if we parenthesized the expression as $(a + b) + c$. All Python operators of the same precedence group left-to-right except for the exponentiation operator (**), which groups right-to-left.

Redundant Parentheses

You can use redundant parentheses to group subexpressions to make the expression clearer. For example, the second-degree polynomial

$$y = a * x ** 2 + b * x + c$$

can be parenthesized, for clarity, as

$$y = (a * (x ** 2)) + (b * x) + c$$

Breaking a complex expression into a sequence of statements with shorter, simpler expressions also can promote clarity.

Operand Types

Each arithmetic operator may be used with integers and floating-point numbers. If both operands are integers, the result is an integer—except for the true-division (/) operator, which always yields a floating-point number. If both operands are floating-point numbers, the result is a floating-point number. Expressions containing an integer and a floating-point number are **mixed-type expressions**—these always produce floating-point numbers.



Self Check

1 (Multiple Choice) Given that $y = ax^3 + 7$, which of the following is not a correct statement for this equation?

- a) $y = a * x * x * x + 7$
- b) $y = a * x ** 3 + 7$
- c) $y = a * (x * x * x) + 7$
- d) $y = a * x * (x * x + 7)$

Answer: d is incorrect.

2 (True/False) In nested parentheses, the expression in the innermost pair evaluates last.

Answer: False. The expression in the innermost parentheses evaluates first.

3 (IPython Session) Evaluate the expression $3 * (4 - 5)$ with and without parentheses. Are the parentheses redundant?

Answer:

```
In [1]: 3 * (4 - 5)
Out[1]: -3
```

```
In [2]: 3 * 4 - 5
Out[2]: 7
```

The parentheses are not redundant—if you remove them the resulting value is different.

4 (*IPython Session*) Evaluate the expressions `4 ** 3 ** 2`, `(4 ** 3) ** 2` and `4 ** (3 ** 2)`. Are any of the parentheses redundant?

Answer:

```
In [3]: 4 ** 3 ** 2
Out[3]: 262144

In [4]: (4 ** 3) ** 2
Out[4]: 4096

In [5]: 4 ** (3 ** 2)
Out[5]: 262144
```

Only the parentheses in the last expression are redundant.

2.4 Function `print` and an Intro to Single- and Double-Quoted Strings

The built-in **`print` function** displays its argument(s) as a line of text:

```
In [1]: print('Welcome to Python!')
Welcome to Python!
```

In this case, the argument `'Welcome to Python!'` is a **string**—a sequence of characters enclosed in single quotes (`'`). Unlike when you evaluate expressions in interactive mode, the text that `print` displays here is not preceded by `Out[1]`. Also, `print` does not display a string's quotes, though we'll soon show how to display quotes in strings.

You also may enclose a string in double quotes (`"`), as in:

```
In [2]: print("Welcome to Python!")
Welcome to Python!
```

Python programmers generally prefer single quotes.

When `print` completes its task, it positions the screen cursor at the beginning of the next line. This is similar to what happens when you press the *Enter* (or *Return*) key while typing in a text editor.

Printing a Comma-Separated List of Items

The `print` function can receive a comma-separated list of arguments, as in:

```
In [3]: print('Welcome', 'to', 'Python!')
Welcome to Python!
```

The `print` function displays each argument separated from the next by a space, producing the same output as in the two preceding snippets. Here we showed a comma-separated list of strings, but the values can be of any type. We'll show in the next chapter how to prevent automatic spacing between values or use a different separator than space.

Printing Many Lines of Text with One Statement

When a backslash (`\`) appears in a string, it's known as the **escape character**. The backslash and the character immediately following it form an **escape sequence**. For example, `\n` represents the **newline character** escape sequence, which tells `print` to move the output cursor to the next line. Placing two newline characters back-to-back displays a blank line. The following snippet uses three newline characters to create many lines of output:

```
In [4]: print('Welcome\nto\n\nPython!')
Welcome
to

Python!
```

Other Escape Sequences

The following table shows some common escape sequences.

Escape sequence	Description
<code>\n</code>	Insert a newline character in a string. When the string is displayed, for each newline, move the screen cursor to the beginning of the next line.
<code>\t</code>	Insert a horizontal tab. When the string is displayed, for each tab, move the screen cursor to the next tab stop.
<code>\\</code>	Insert a backslash character in a string.
<code>\"</code>	Insert a double quote character in a string.
<code>\'</code>	Insert a single quote character in a string.

Ignoring a Line Break in a Long String

You may also split a long string (or a long statement) over several lines by using the `\ continuation character` as the last character on a line to ignore the line break:

```
In [5]: print('this is a longer string, so we \
...: split it over two lines')
this is a longer string, so we split it over two lines
```

The interpreter reassembles the string's parts into a single string with no line break. Though the backslash character in the preceding snippet is inside a string, it's not the escape character because another character does not follow it.

Printing the Value of an Expression

Calculations can be performed in print statements:

```
In [6]: print('Sum is', 7 + 3)
Sum is 10
```



Self Check

1 (*Fill-In*) The _____ function instructs the computer to display information on the screen.

Answer: print.

2 (*Fill-In*) Values of the _____ data type contain a sequence of characters.

Answer: string (type str).

3 (*IPython Session*) Write an expression that displays the type of 'word'.

Answer:

```
In [1]: type('word')
Out[1]: str
```

4 (*IPython Session*) What does the following print statement display?

```
print('int(5.2)', 'truncates 5.2 to', int(5.2))
```

Answer:

```
In [2]: print('int(5.2)', 'truncates 5.2 to', int(5.2))
int(5.2) truncates 5.2 to 5
```

2.5 Triple-Quoted Strings

Earlier, we introduced strings delimited by a pair of single quotes (') or a pair of double quotes ("). **Triple-quoted strings** begin and end with three double quotes ("""") or three single quotes (''''). The *Style Guide for Python Code* recommends three double quotes ("""). Use these to create:

- multiline strings,
- strings containing single or double quotes and
- **docstrings**, which are the recommended way to document the purposes of certain program components.

Including Quotes in Strings

In a string delimited by single quotes, you may include double-quote characters:

```
In [1]: print('Display "hi" in quotes')
Display "hi" in quotes
```

but not single quotes:

```
In [2]: print('Display 'hi' in quotes')
File "<ipython-input-2-19bf596ccf72>", line 1
    print('Display 'hi' in quotes')
                   ^
SyntaxError: invalid syntax
```

unless you use the \ ' escape sequence:

```
In [3]: print('Display \'hi\' in quotes')
Display 'hi' in quotes
```

Snippet [2] displayed a **syntax error**, which is a violation of Python's language rules—in this case, a single quote inside a single-quoted string. IPython displays information about the line of code that caused the syntax error and points to the error with a ^ symbol. It also displays the message `SyntaxError: invalid syntax`.

A string delimited by double quotes may include single quote characters:

```
In [4]: print("Display the name O'Brien")
Display the name O'Brien
```

but not double quotes, unless you use the \" escape sequence:

```
In [5]: print("Display \"hi\" in quotes")
Display "hi" in quotes
```

To avoid using \ ' and \" inside strings, you can enclose such strings in triple quotes:

```
In [6]: print("""Display "hi" and 'bye' in quotes""")
Display "hi" and 'bye' in quotes
```

Multiline Strings

The following snippet assigns a multiline triple-quoted string to `triple_quoted_string`:

```
In [7]: triple_quoted_string = """This is a triple-quoted
...: string that spans two lines"""
```

IPython knows that the string is incomplete because we did not type the closing `"""` before we pressed *Enter*. So, IPython displays a **continuation prompt** `...:` at which you can input the multiline string's next line. This continues until you enter the ending `"""` and press *Enter*. The following displays `triple_quoted_string`:

```
In [8]: print(triple_quoted_string)
This is a triple-quoted
string that spans two lines
```

Python stores multiline strings with embedded newline escape sequences. When we evaluate `triple_quoted_string` rather than printing it, IPython displays the string in single quotes with a `\n` character where you pressed *Enter* in snippet [7]. The quotes IPython displays indicate that `triple_quoted_string` is a string—they're not part of the string's contents:

```
In [9]: triple_quoted_string
Out[9]: 'This is a triple-quoted\nstring that spans two lines'
```



Self Check

1 (Fill-In) Multiline strings are enclosed either in _____ or in _____.

Answer: `"""` (triple double quotes) or `'''` (triple single quotes).

2 (IPython Session) What displays when you execute the following statement?

```
print("""This is a lengthy
      multiline string containing
      a few lines \
of text""")
```

Answer:

```
In [1]: print("""This is a lengthy
...:      multiline string containing
...:      a few lines \
...:      of text""")
This is a lengthy
multiline string containing
a few lines of text
```

2.6 Getting Input from the User

The built-in **input function** requests and obtains user input:

```
In [1]: name = input("What's your name? ")
What's your name? Paul

In [2]: name
Out[2]: 'Paul'

In [3]: print(name)
Paul
```

The snippet executes as follows:

- First, `input` displays its string argument—called a **prompt**—to tell the user what to type and waits for the user to respond. We typed `Paul` (without quotes) and pressed *Enter*. We use **bold** text to distinguish the user’s input from the prompt text that `input` displays.
- Function `input` then **returns** (that is, gives back) those characters as a string that the program can use. Here we assigned that string to the variable `name`.

Snippet [2] shows `name`’s value. Evaluating `name` displays its value in single quotes as `'Paul'` because it’s a string. Printing `name` (in snippet [3]) displays the string without the quotes. If you enter quotes, they’re part of the string, as in:

```
In [4]: name = input("What's your name? ")
What's your name? 'Paul'

In [5]: name
Out[5]: "'Paul'"

In [6]: print(name)
'Paul'
```

Function `input` Always Returns a String

Consider the following snippets that attempt to read two numbers and add them:

```
In [7]: value1 = input('Enter first number: ')
Enter first number: 7

In [8]: value2 = input('Enter second number: ')
Enter second number: 3

In [9]: value1 + value2
Out[9]: '73'
```

Rather than adding the integers 7 and 3 to produce 10, Python “adds” the *string* values `'7'` and `'3'`, producing the *string* `'73'`. This is known as **string concatenation**. It creates a new string containing the left operand’s value followed by the right operand’s value.

Getting an Integer from the User

If you need an integer, convert the string to an integer using the built-in **`int` function**:

```
In [10]: value = input('Enter an integer: ')
Enter an integer: 7

In [11]: value = int(value)

In [12]: value
Out[12]: 7
```

We could have combined the code in snippets [10] and [11]:

```
In [13]: another_value = int(input('Enter another integer: '))
Enter another integer: 13

In [14]: another_value
Out[14]: 13
```

Variables `value` and `another_value` now contain integers. Adding them produces an integer result (rather than concatenating them):

```
In [15]: value + another_value
Out[15]: 20
```

If the string passed to `int` cannot be converted to an integer, a `ValueError` occurs:

```
In [16]: bad_value = int(input('Enter another integer: '))
Enter another integer: hello

-----
ValueError                                Traceback (most recent call last)
<ipython-input-16-cd36e6cf8911> in <module>()
----> 1 bad_value = int(input('Enter another integer: '))

ValueError: invalid literal for int() with base 10: 'hello'
```

Function `int` also can convert a floating-point value to an integer:

```
In [17]: int(10.5)
Out[17]: 10
```

To convert strings to floating-point numbers, use the built-in **float function**.



Self Check

1 (Fill-In) The built-in _____ function converts a floating-point value to an integer value or converts a string representation of an integer to an integer value.
Answer: `int`.

2 (True/False) Built-in function `get_input` requests and obtains input from the user.
Answer: False. The built-in function's name is `input`.

3 (IPython Session) Use `float` to convert `'6.2'` (a string) to a floating-point value. Multiply that value by `3.3` and show the result.
Answer:

```
In [1]: float('6.2') * 3.3
Out[1]: 20.46
```

2.7 Decision Making: The if Statement and Comparison Operators

A **condition** is a Boolean expression with the value **True** or **False**. The following determines whether 7 is greater than 4 and whether 7 is less than 4:

```
In [1]: 7 > 4
Out[1]: True

In [2]: 7 < 4
Out[2]: False
```

`True` and `False` are **keywords**—words that Python reserves for its language features. Using a keyword as an identifier causes a `SyntaxError`. `True` and `False` are each capitalized.

You'll often create conditions using the **comparison operators** in the table at the top of the next page:

Algebraic operator	Python operator	Sample condition	Meaning
>	>	$x > y$	x is greater than y
<	<	$x < y$	x is less than y
$\geq$	<code>>=</code>	$x \geq y$	x is greater than or equal to y
$\leq$	<code><=</code>	$x \leq y$	x is less than or equal to y
=	<code>==</code>	$x == y$	x is equal to y
$\neq$	<code>!=</code>	$x != y$	x is not equal to y

Operators `>`, `<`, `>=` and `<=` all have the same precedence. Operators `==` and `!=` both have the same precedence, which is lower than that of `>`, `<`, `>=` and `<=`. A syntax error occurs when any of the operators `==`, `!=`, `>=` and `<=` contains spaces between its pair of symbols:

```
In [3]: 7 > = 4
         File "<ipython-input-3-5c6e2897f3b3>", line 1
           7 > = 4
             ^
SyntaxError: invalid syntax
```

Another syntax error occurs if you reverse the symbols in the operators `!=`, `>=` and `<=` (by writing them as `=!`, `=>` and `=<`).

Making Decisions with the `if` Statement: Introducing Scripts

We now present a simple version of the **`if` statement**, which uses a condition to decide whether to execute a statement (or a group of statements). Here we'll read two integers from the user and compare them using six consecutive `if` statements, one for each comparison operator. If the condition in a given `if` statement is `True`, the corresponding `print` statement executes; otherwise, it's skipped.

IPython interactive mode is helpful for executing brief code snippets and seeing immediate results. When you have many statements to execute as a group, you typically write them as a **script** stored in a file with the `.py` (short for Python) extension—such as `fig02_01.py` for this example's script. Scripts are also called **programs**. For instructions on locating and executing the scripts in this book, see Chapter 1's IPython Test-Drive.

Each time you execute this script, three of the six conditions are `True`. To show this, we execute the script three times—once with the first integer *less than* the second, once with the *same* value for both integers and once with the first integer *greater than* the second. The three sample executions appear after the script

Figure 2.1 shows the script. Each time we present a script, we introduce it before the figure, then explain the script's code after the figure. We show line numbers for your convenience—these are not part of Python. Integrated development environments (IDEs) enable you to choose whether to display line numbers. To run this example, change to this chapter's `ch02 examples` folder, then enter:

```
ipython fig02_01.py
```

or, if you're in IPython already, use the command:

```
run fig02_01.py
```

```
1 # fig02_01.py
2 """Comparing integers using if statements and comparison operators."""
3
4 print('Enter two integers, and I will tell you',
5       'the relationships they satisfy.')
6
7 # read first integer
8 number1 = int(input('Enter first integer: '))
9
10 # read second integer
11 number2 = int(input('Enter second integer: '))
12
13 if number1 == number2:
14     print(number1, 'is equal to', number2)
15
16 if number1 != number2:
17     print(number1, 'is not equal to', number2)
18
19 if number1 < number2:
20     print(number1, 'is less than', number2)
21
22 if number1 > number2:
23     print(number1, 'is greater than', number2)
24
25 if number1 <= number2:
26     print(number1, 'is less than or equal to', number2)
27
28 if number1 >= number2:
29     print(number1, 'is greater than or equal to', number2)
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 37
Enter second integer: 42
37 is not equal to 42
37 is less than 42
37 is less than or equal to 42
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 7
Enter second integer: 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 54
Enter second integer: 17
54 is not equal to 17
54 is greater than 17
54 is greater than or equal to 17
```

Fig. 2.1 | Comparing integers using if statements and comparison operators.

Comments

Line 1 begins with the hash character (`#`), which indicates that the rest of the line is a **comment**:

```
# fig02_01.py
```

You insert comments to document your code and to improve readability. Comments also help other programmers read and understand your code. They do not cause the computer to perform any action when the code executes. For easy reference, we begin each script with a comment indicating the script's file name.

A comment also can begin to the right of the code on a given line and continue until the end of that line. Such a comment documents the code to its left.

Docstrings

The *Style Guide for Python Code* states that each script should start with a docstring that explains the script's purpose, such as the one in line 2:

```
"""Comparing integers using if statements and comparison operators."""
```

For more complex scripts, the docstring often spans many lines. In later chapters, you'll use docstrings to describe script components you define, such as new functions and new types called classes. We'll also discuss how to access docstrings with the IPython help mechanism.

Blank Lines

Line 3 is a blank line. You use blank lines and space characters to make code easier to read. Together, blank lines, space characters and tab characters are known as **white space**. Python ignores most white space—you'll see that some indentation is required.

Splitting a Lengthy Statement Across Lines

Lines 4–5

```
print('Enter two integers, and I will tell you',  
      'the relationships they satisfy.')
```

display instructions to the user. These are too long to fit on one line, so we broke them into two strings. Recall that you can display several values by passing to `print` a comma-separated list—`print` separates each value from the next with a space character.

Typically, you write statements on one line. You may spread a lengthy statement over several lines with the `\` continuation character. Python also allows you to split long code lines in parentheses without using continuation characters (as in lines 4–5). This is the preferred way to break long code lines according to the *Style Guide for Python Code*. Always choose breaking points that make sense, such as after a comma in the preceding call to `print` or before an operator in a lengthy expression.

Reading Integer Values from the User

Next, lines 8 and 11 use the built-in `input` and `int` functions to prompt for and read two integer values from the user.

`if` Statements

The `if` statement in lines 13–14

```
if number1 == number2:
    print(number1, 'is equal to', number2)
```

uses the `==` comparison operator to determine whether the values of variables `number1` and `number2` are equal. If so, the condition is `True`, and line 14 displays a line of text indicating that the values are equal. If any of the remaining `if` statements' conditions are `True` (lines 16, 19, 22, 25 and 28), the corresponding `print` displays a line of text.

Each `if` statement consists of the keyword `if`, the condition to test, and a colon (`:`) followed by an indented body called a **suite**. Each suite must contain one or more statements. Forgetting the colon (`:`) after the condition is a common syntax error.

Suite Indentation

Python requires you to indent the statements in suites. The *Style Guide for Python Code* recommends four-space indents—we use that convention throughout this book. You'll see in the next chapter that incorrect indentation can cause errors.

Confusing `==` and `=`

Using the assignment symbol (`=`) instead of the equality operator (`==`) in an `if` statement's condition is a common syntax error. To help avoid this, read `==` as “is equal to” and `=` as “is assigned.” You'll see in the next chapter that using `==` in place of `=` in an assignment statement can lead to subtle problems.

Chaining Comparisons

You can chain comparisons to check whether a value is in a range. The following comparison determines whether `x` is in the range 1 through 5, inclusive:

```
In [1]: x = 3

In [2]: 1 <= x <= 5
Out[2]: True

In [3]: x = 10

In [4]: 1 <= x <= 5
Out[4]: False
```

Precedence of the Operators We've Presented So Far

The precedence of the operators introduced in this chapter is shown below:

Operators	Grouping	Type
<code>()</code>	left to right	parentheses
<code>**</code>	right to left	exponentiation
<code>*</code> <code>/</code> <code>//</code> <code>%</code>	left to right	multiplication, true division, floor division, remainder
<code>+</code> <code>-</code>	left to right	addition, subtraction
<code>></code> <code><=</code> <code><</code> <code>>=</code>	left to right	less than, less than or equal, greater than, greater than or equal
<code>==</code> <code>!=</code>	left to right	equal, not equal

The table lists the operators top-to-bottom in decreasing order of precedence. When writing expressions containing multiple operators, confirm that they evaluate in the order you expect by referring to the operator precedence chart at

<https://docs.python.org/3/reference/expressions.html#operator-precedence>



Self Check

1 (Fill-In) You use _____ to document code and improve its readability.

Answer: comments.

2 (True/False) The comparison operators evaluate left to right and all have the same level of precedence.

Answer: False. The operators <, <=, > and >= all have the same level of precedence and evaluate left to right. The operators == and != have the same level of precedence and evaluate left to right. Their precedence is lower than that of <, <=, > and >=.

3 (IPython Session) For any of the operators !=, >= or <=, show that a syntax error occurs if you reverse the symbols in a condition.

Answer:

```
In [1]: 7 =< 10
File "<ipython-input-1-090d4004a38e>", line 1
      7 =< 10
        ^
SyntaxError: invalid syntax
```

4 (IPython Session) Use all six comparison operators to compare the values 5 and 9. Display the values on one line using print.

Answer:

```
In [2]: print(5 < 9, 5 <= 9, 5 > 9, 5 >= 9, 5 == 9, 5 != 9)
True True False False False True
```

2.8 Objects and Dynamic Typing

The first chapter introduced the terms *classes* and *objects* and in Section 2.2, we discussed variables, values and types. Values such as 7 (an integer), 4.1 (a floating-point number) and 'dog' are all objects. Every object has a type and a value:

```
In [1]: type(7)
Out[1]: int

In [2]: type(4.1)
Out[2]: float

In [3]: type('dog')
Out[3]: str
```

An object's value is the data stored in the object. The snippets above show objects of Python built-in types **int** (for integers), **float** (for floating-point numbers) and **str** (for strings).

Variables Refer to Objects

Assigning an object to a variable **binds** (associates) that variable's name to the object. As you've seen, you can then use the variable in your code to access the object's value:

```
In [4]: x = 7

In [5]: x + 10
Out[5]: 17

In [6]: x
Out[6]: 7
```

After snippet [4]’s assignment, the variable `x` **refers** to the integer object containing 7. As shown in snippet [6], snippet [5] does not change `x`’s value. You can change `x` as follows:

```
In [7]: x = x + 10

In [8]: x
Out[8]: 17
```

Dynamic Typing

Python uses **dynamic typing**—it determines the type of the object a variable refers to while executing your code. We can show this by rebinding the variable `x` to different objects and checking their types:

```
In [9]: type(x)
Out[9]: int

In [10]: x = 4.1

In [11]: type(x)
Out[11]: float

In [12]: x = 'dog'

In [13]: type(x)
Out[13]: str
```

Garbage Collection

Python creates objects in memory and removes them from memory as necessary. After snippet [10], the variable `x` now refers to a `float` object. The integer object from snippet [7] is no longer bound to a variable. As we’ll discuss in a later chapter, Python automatically removes such objects from memory. This process—called **garbage collection**—helps ensure that memory is available for new objects you create.



Self Check

1 (Fill-In) Assigning an object to a variable _____ the variable’s name to the object.
Answer: binds.

2 (True/False) A variable always references the same object.
Answer: False. You can make an existing variable refer to a different object and even one of a different type.

3 (IPython Session) What is the type of the expression `7.5 * 3`?
Answer:

```
In [1]: type(7.5 * 3)
Out[1]: float
```

2.9 Intro to Data Science: Basic Descriptive Statistics

In data science, you'll often use statistics to describe and summarize your data. Here, we begin by introducing several such **descriptive statistics**, including:

- **minimum**—the smallest value in a collection of values.
- **maximum**—the largest value in a collection of values.
- **range**—the range of values from the minimum to the maximum.
- **count**—the number of values in a collection.
- **sum**—the total of the values in a collection.

We'll look at determining the *count* and *sum* in the next chapter. **Measures of dispersion** (also called **measures of variability**), such as *range*, help determine how spread out values are. Other measures of dispersion that we'll present in later chapters include *variance* and *standard deviation*.

Determining the Minimum of Three Values

First, let's show how to determine the minimum of three values manually. The following script prompts for and inputs three values, uses `if` statements to determine the minimum value, then displays it.

```

1  # fig02_02.py
2  """Find the minimum of three values."""
3
4  number1 = int(input('Enter first integer: '))
5  number2 = int(input('Enter second integer: '))
6  number3 = int(input('Enter third integer: '))
7
8  minimum = number1
9
10 if number2 < minimum:
11     minimum = number2
12
13 if number3 < minimum:
14     minimum = number3
15
16 print('Minimum value is', minimum)
```

```

Enter first integer: 12
Enter second integer: 27
Enter third integer: 36
Minimum value is 12
```

```

Enter first integer: 27
Enter second integer: 12
Enter third integer: 36
Minimum value is 12
```

Fig. 2.2 | Find the minimum of three values. (Part 1 of 2.)

```
Enter first integer: 36
Enter second integer: 27
Enter third integer: 12
Minimum value is 12
```

Fig. 2.2 | Find the minimum of three values. (Part 2 of 2.)

After inputting the three values, we process one value at a time:

- First, we assume that `number1` contains the smallest value, so line 8 assigns it to the variable `minimum`. Of course, it's possible that `number2` or `number3` contains the actual smallest value, so we still must compare each of these with `minimum`.
- The first `if` statement (lines 10–11) then tests `number2 < minimum` and if this condition is `True` assigns `number2` to `minimum`.
- The second `if` statement (lines 13–14) then tests `number3 < minimum`, and if this condition is `True` assigns `number3` to `minimum`.

Now, `minimum` contains the smallest value, so we display it. We executed the script three times to show that it always finds the smallest value regardless of whether the user enters it first, second or third.

Determining the Minimum and Maximum with Built-In Functions `min` and `max`

Python has many built-in functions for performing common tasks. Built-in functions `min` and `max` calculate the minimum and maximum, respectively, of a collection of values:

```
In [1]: min(36, 27, 12)
Out[1]: 12

In [2]: max(36, 27, 12)
Out[2]: 36
```

The functions `min` and `max` can receive any number of arguments.

Determining the Range of a Collection of Values

The *range* of values is simply the minimum through the maximum value. In this case, the range is 12 through 36. Much data science is devoted to getting to know your data. Descriptive statistics is a crucial part of that, but you also have to understand how to interpret the statistics. For example, if you have 100 numbers with a range of 12 through 36, those numbers could be distributed evenly over that range. At the opposite extreme, you could have clumping with 99 values of 12 and one 36, or one 12 and 99 values of 36. In later data science sections, we'll look at common *data distributions*.

Functional-Style Programming: Reduction

Throughout this book, we introduce various *functional-style programming* capabilities. These enable you to write code that can be more concise, clearer and easier to **debug**—that is, find and correct errors. The `min` and `max` functions are examples of a functional-style programming concept called **reduction**. They reduce a collection of values to a *single* value. Other reductions you'll see include the sum, average, variance and standard deviation of a collection of values. You'll also learn how to define custom reductions.

Upcoming Intro to Data Science Sections

In the next two chapters, we'll continue our discussion of basic descriptive statistics with *measures of central tendency*, including *mean*, *median* and *mode*, and *measures of dispersion*, including *variance* and *standard deviation*.



Self Check

1 (*Fill-In*) The range of a collection of values is a measure of _____.

Answer: dispersion.

2 (*IPython Session*) For the values 47, 95, 88, 73, 88 and 84 calculate the minimum, maximum and range.

Answer:

```
In [1]: min(47, 95, 88, 73, 88, 84)
Out[1]: 47

In [2]: max(47, 95, 88, 73, 88, 84)
Out[2]: 95

In [3]: print('Range:', min(47, 95, 88, 73, 88, 84), '-',
...:         max(47, 95, 88, 73, 88, 84))
...:
...:
Range: 47 - 95
```

2.10 Wrap-Up

This chapter continued our discussion of arithmetic. You used variables to store values for later use. We introduced Python's arithmetic operators and showed that you must write all expressions in straight-line form. You used the built-in function `print` to display data. We created single-, double- and triple-quoted strings. You used triple-quoted strings to create multiline strings and to embed single or double quotes in strings.

You used the `input` function to prompt for and get input from the user at the keyboard. We used the functions `int` and `float` to convert strings to numeric values. We presented Python's comparison operators. Then, you used them in a script that read two integers from the user and compared their values using a series of `if` statements.

We discussed Python's dynamic typing and used the built-in function `type` to display an object's type. Finally, we introduced the basic descriptive statistics minimum and maximum and used them to calculate the range of a collection of values. In the next chapter, you'll learn Python's control statements and program development.

Exercises

Unless specified otherwise, use IPython sessions for each exercise.

2.1 (*What does this code do?*) Create the variables `x = 2` and `y = 3`, then determine what each of the following statements displays:

- `print('x =', x)`
- `print('Value of', x, '+', x, 'is', (x + x))`
- `print('x =')`
- `print((x + y), '=', (y + x))`

2.2 (*What's wrong with this code?*) The following code should read an integer into the variable `rating`:

```
rating = input('Enter an integer rating between 1 and 10')
```

2.3 (*Fill in the missing code*) Replace `***` in the following code with a statement that will print a message like 'Congratulations! Your grade of 91 earns you an A in this course'. Your statement should print the value stored in the variable `grade`:

```
if grade >= 90:
    ***
```

2.4 (*Arithmetic*) For each of the arithmetic operators `+`, `-`, `*`, `/`, `//` and `**`, display the value of an expression with `27.5` as the left operand and `2` as the right operand.

2.5 (*Circle Area, Diameter and Circumference*) For a circle of radius `2`, display the diameter, circumference and area. Use the value `3.14159` for π . Use the following formulas (r is the radius): *diameter* = $2r$, *circumference* = $2\pi r$ and *area* = πr^2 . [In a later chapter, we'll introduce Python's `math` module which contains a higher-precision representation of π .]

2.6 (*Odd or Even*) Use `if` statements to determine whether an integer is odd or even. [Hint: Use the remainder operator. An even number is a multiple of `2`. Any multiple of `2` leaves a remainder of `0` when divided by `2`.]

2.7 (*Multiples*) Use `if` statements to determine whether `1024` is a multiple of `4` and whether `2` is a multiple of `10`. (Hint: Use the remainder operator.)

2.8 (*Table of Squares and Cubes*) Write a script that calculates the squares and cubes of the numbers from `0` to `5`. Print the resulting values in table format, as shown below. Use the tab escape sequence to achieve the three-column output.

number	square	cube
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125

The next chapter shows how to “right align” numbers. You could try that as an extra challenge here. The output would be:

number	square	cube
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125

2.9 (*Integer Value of a Character*) Here's a peek ahead. In this chapter, you learned about strings. Each of a string's characters has an integer representation. The set of characters a computer uses together with the characters' integer representations is called that computer's *character set*. You can indicate a character value in a program by enclosing that character in quotes, as in `'A'`. To determine a character's integer value, call the built-in function `ord`:

```
In [1]: ord('A')
Out[1]: 65
```

Display the integer equivalents of B C D b c d 0 1 2 \$ * + and the space character.

2.10 (*Arithmetic, Smallest and Largest*) Write a script that inputs three integers from the user. Display the sum, average, product, smallest and largest of the numbers. Note that each of these is a reduction in functional-style programming.

2.11 (*Separating the Digits in an Integer*) Write a script that inputs a five-digit integer from the user. Separate the number into its individual digits. Print them separated by three spaces each. For example, if the user types in the number 42339, the script should print

```
4    2    3    3    9
```

Assume that the user enters the correct number of digits. Use both the floor division and remainder operations to “pick off” each digit.

2.12 (*7% Investment Return*) Some investment advisors say that it’s reasonable to expect a 7% return over the long term in the stock market. Assuming that you begin with \$1000 and leave your money invested, calculate and display how much money you’ll have after 10, 20 and 30 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal of \$1000),

r is the annual rate of return (7%),

n is the number of years (10, 20 or 30) and

a is the amount on deposit at the end of the n th year.

2.13 (*How Big Can Python Integers Be?*) We’ll answer this question later in the book. For now, use the exponentiation operator `**` with large and very large exponents to produce some huge integers and assign those to the variable `number` to see if Python accepts them. Did you find any integer value that Python won’t accept?

2.14 (*Target Heart-Rate Calculator*) While exercising, you can use a heart-rate monitor to see that your heart rate stays within a safe range suggested by your doctors and trainers. According to the American Heart Association (AHA) (<http://bit.ly/AHATargetHeartRates>), the formula for calculating your maximum heart rate in beats per minute is 220 minus your age in years. Your target heart rate is 50–85% of your maximum heart rate. Write a script that prompts for and inputs the user’s age and calculates and displays the user’s maximum heart rate and the range of the user’s target heart rate. [These formulas are estimates provided by the AHA; maximum and target heart rates may vary based on the health, fitness and gender of the individual. Always consult a physician or qualified healthcare professional before beginning or modifying an exercise program.]

2.15 (*Sort in Ascending Order*) Write a script that inputs three different floating-point numbers from the user. Display the numbers in increasing order. Recall that an `if` statement’s suite can contain more than one statement. Prove that your script works by running it on all six possible orderings of the numbers. Does your script work with duplicate numbers? [This is challenging. In later chapters you’ll do this more conveniently and with many more numbers.]